

LC 2547: Minimum Cost to Split an Array

Mathematical & Conceptual Notes

- We need to split an array and calculate the minimum cost.
- **Cost** = Sum of importance value.
- To understand the importance value, it is defined as:
 $k + \text{trimmed value}$
- **Example:** $[1, 2, 3, 3, 3, 4, 4] \rightarrow [3, 3, 3, 4, 4]$ (trimmed)
Basically, remove all elements with frequency 1.
So the importance value here would be $k + 5$.
- **Example Split:** Array $[1, 2, 1, 2, 1, 3, 3]$, with $k = 2$.
Let the split be: $1, 2 \mid 1, 2, 1, 3, 3$
Then the cost is: $(k + 0) + (k + 4) = 8$.

Algorithm Strategy

- Precompute the cost for all subarrays: $\text{cost}[l][r]$.
- Use an `unordered_map` to track frequencies.
- Fix l , and move r to populate the cost matrix.
- Use $\text{dp}[i] = \text{min cost to split the first } i \text{ elements, as the number of parts isn't fixed}$.

| Approach 1: Partition DP (Result: TLE)

```

class Solution {
public:
    int minCost(vector<int>& nums, int k) {
        int n = nums.size();

        // cost precompute kr lenge
        vector<vector<int>> cost(n, vector<int>(n, 0));
        // cost[l][r]

        for(int l = 0; l < n; l++) {
            unordered_map<int, int> freq;
            int tcost = 0;

            for(int r = l; r < n; r++) {
                int f = freq[nums[r]];
                if(f == 1) {
                    // already ek baar aaya he
                    tcost += 2;
                }
                else if(f >= 2) {
                    tcost++;
                }
                freq[nums[r]]++;
                cost[l][r] = tcost;
            }
        }

        // partition dp
        // dp[i][k]
        vector<vector<int>> dp(n + 1, vector<int>(n + 1, INT_MAX));
        dp[0][0] = 0;

        for(int i = 1; i <= n; i++) {
            for(int parts = 1; parts <= i; parts++) { // k is not number of parts but ju
                                                        // so number of parts can go up to
                for(int j = parts - 1; j < i; j++) {
                    if(dp[j][parts - 1] == INT_MAX) continue;
                    int curr = dp[j][parts - 1] + k + cost[j][i - 1];
                    dp[i][parts] = min(dp[i][parts], curr);
                }
            }
        }

        // can be any number of parts, need minimum
        int ans = INT_MAX;
        for(int parts = 1; parts <= n; parts++) {
            ans = min(ans, dp[n][parts]);
        }

        return ans;
    }
};

```

```
}  
};
```

Optimization Idea:

This gave TLE. The optimization idea is to use only $dp[i]$ and ignore parts completely. It will give the minimum cost to split the first i elements.

Approach 2: Optimized 1D DP

```
class Solution {
public:
    int minCost(vector<int>& nums, int k) {
        int n = nums.size();

        // cost precompute kr lenge
        vector<vector<int>>> cost(n, vector<int>(n, 0));
        // cost[l][r]

        for(int l = 0; l < n; l++) {
            unordered_map<int, int> freq;
            int tcost = 0;

            for(int r = l; r < n; r++) {
                int f = freq[nums[r]];
                if(f == 1) {
                    // already ek baar aaya he
                    tcost += 2;
                }
                else if(f >= 2) {
                    tcost++;
                }
                freq[nums[r]]++;
                cost[l][r] = tcost;
            }
        }

        // partition dp
        // dp[i] = min cost for first i elements
        vector<int> dp(n + 1, INT_MAX);
        dp[0] = 0;

        for(int i = 1; i <= n; i++) {
            for(int j = i - 1; j >= 0; j--) {
                if(dp[j] == INT_MAX) continue;
                int curr = dp[j] + k + cost[j][i - 1];
                dp[i] = min(dp[i], curr);
            }
        }

        return dp[n];
    }
};
```